

Task 1: Create my-music-uploads

Steps

1. Open Amazon S3.
2. Click Create bucket.
3. Enter:
 - my-music-uploads-nishit-2026
4. Select Asia Pacific (Mumbai).
5. Keep Block Public Access enabled.
6. Create the bucket.
7. Upload two MP3 files.

Example:

- spotify-song1.mp3
- spotify-song2.mp3

Task 2: Restrict Download Access

A safer AWS approach is to use an IAM user/role rather than trying to directly put an email address into an S3 bucket policy.

Example bucket policy

Replace:

YOUR-BUCKET-NAME

with your actual bucket name.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSpecificUserReadOnly",
      "Effect": "Allow",
      "Principal": {
```

```

        "AWS": "arn:aws:iam::ACCOUNT-ID:user/YOUR-IAM-USER"
    },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
}
]
}

```

Task 3: Enable Server-Side Encryption

Steps

1. Open the S3 bucket.
2. Go to Properties.
3. Find Default encryption.
4. Select Server-side encryption with Amazon S3 managed keys (SSE-S3).
5. Save the settings.

Task 4: Test Unauthorized Access

1. Copy the Object URL of an MP3 file.
2. Open an incognito/private browser window where you are not authenticated to AWS.
3. Paste the URL.
4. Attempt to access the object.

If public access is blocked, the browser should not be able to retrieve the object.

Task 5: IP-Based Bucket Policy

Example policy allowing access only from a specific public IP address:

```
{
```

```
"Version": "2012-10-17",
"Statement": [
  {
    "Sid": "AllowOnlyFromSpecificIP",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*",
    "Condition": {
      "IpAddress": {
        "aws:SourceIp": "203.0.113.10/32"
      }
    }
  }
]
```